

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ  
Національний аерокосмічний університет ім. М. Є. Жуковського  
«Харківський авіаційний інститут»

Кафедра інтелектуальних систем управління

Практична робота №3

з дисципліни «Об’єктно-орієнтоване програмування СУ»

Тема: «Структурування програм з використанням функцій»

ХАІ.301 . G3 «Електроінженерія». 319А-ст. 3 ПР

Виконав студент гр. \_\_\_\_\_ 319-аст \_\_\_\_\_

\_\_\_\_\_ 26.09.2025 Кузнєцов Ілля Сергійович  
(підпис, дата) (П.І.Б.)

Перевірів

\_\_\_\_\_ к.т.н., доц. О. В. Гавриленко  
\_\_\_\_\_ ас. В. О. Білозерський  
(підпис, дата) (П.І.Б.)

2025

## МЕТА РОБОТИ

Вивчити теоретичний матеріал із синтаксису визначення і виклику функцій та особливостей послідовностей у Python, а також документацію бібліотеки numpy; отримати навички реалізації бібліотеки функцій з параметрами, що структурують вирішення завдань «згори – до низу».

Завдання 1. Описати функцію Minmax (X, Y), що записує в змінну X мінімальне зі значень X і Y, а в змінну Y - максимальне з цих значень (X і Y — речові параметри, які є одночасно вхідними та вихідними). Використовуючи чотири виклики цієї функції, знайти мінімальне і максимальне з даних чисел A, B, C, D.

Завдання 2. У текстовому файлі задана матриця розміру  $M \times N$  і введено ціле число K ( $1 \leq K \leq M$ ). Знайти суму і добуток елементів K-го рядка даної матриці. Знайти різницю заданої матриці з одиничною матрицею того ж розміру.

Завдання 3. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок за допомогою наступних промптів:

1) «Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції»

2) «Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у 5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку»

## ВИКОНАННЯ ЗАВДАННЯ

Вирішення задачі Proc\_11.

Вхідні дані:

A — дійсне число, float,  $(-\infty; +\infty)$ .

B — дійсне число, float,  $(-\infty; +\infty)$ .

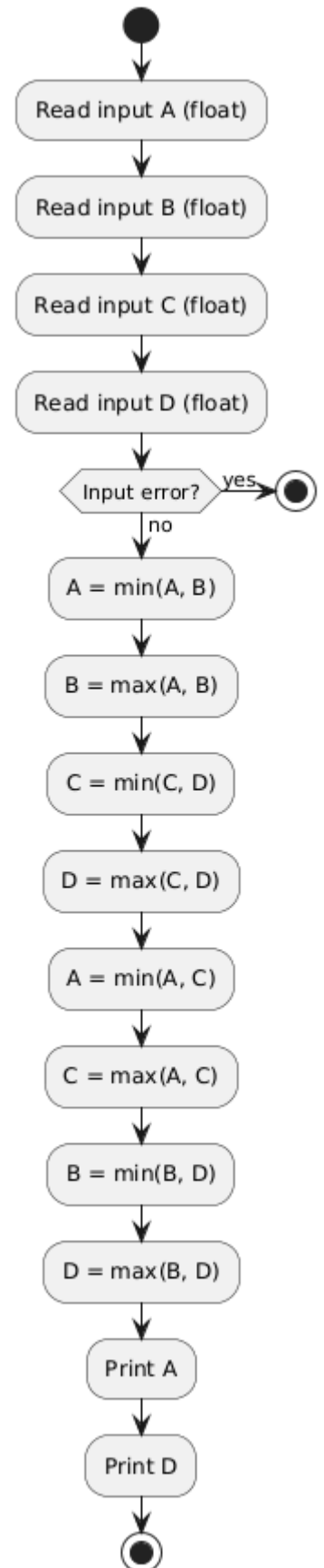
C — дійсне число, float,  $(-\infty; +\infty)$ .

D — дійсне число, float,  $(-\infty; +\infty)$ .

Вихідні дані:

Алгоритм вирішення:

Рисунок 1 - Алгоритм вирішення задачі Proc\_11.



Вирішення задачі Matrix\_1.

Вхідні дані:

M — ціле число, integer,  $(-\infty; +\infty)$ .

K — ціле число, integer,  $(-\infty; +\infty)$ .

N — ціле число, integer,  $(-\infty; +\infty)$ .

Вихідні дані:

Алгоритм вирішення:

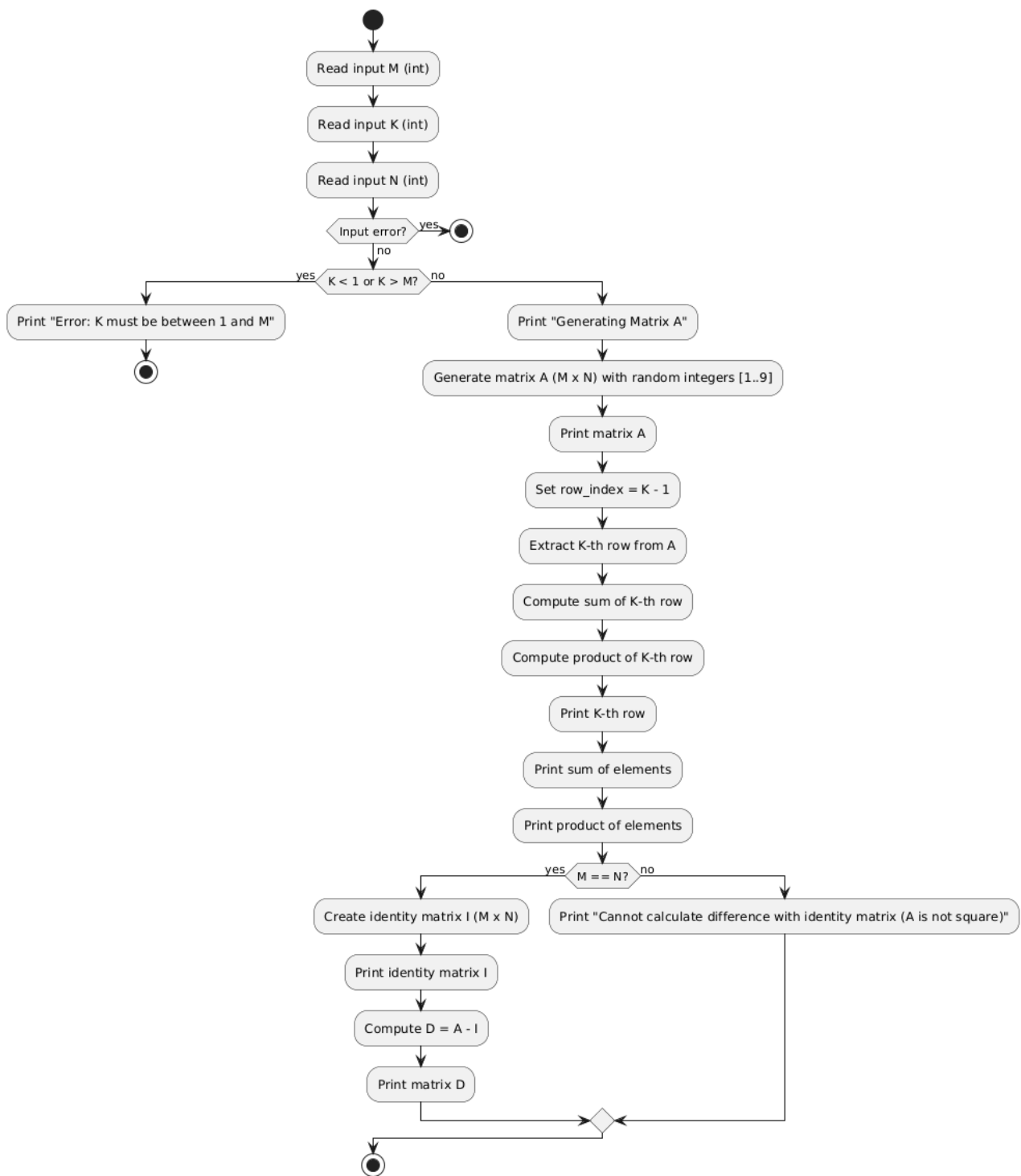


Рисунок 2 - Алгоритм вирішення задачі Matrix\_1.

### Завдання 5: Використовується модель Gemini 3 Flash

1. Припустимо, у вас є глобальна змінна `total = 0` і функція `sum(a, b)`, всередині якої визначено `total = a + b`. Що буде виведено на екран, якщо викликати `sum(10, 20)`, а потім `print(total)` поза функцією?

А. None

Б. Помилка (NameError)

В. 0. Оскільки всередині функції створюється локальна змінна з таким самим ім'ям, глобальна змінна залишається незмінною.

Г. 30

2. Яка з наступних операцій над кортежем `tup = (1, 2, [3, 4])` призведе до помилки (TypeError)?

А. `new_tup = tup + (5, 6)`

Б. `tup[0] = 10` Кортежі є незмінними (immutable) структурами, тому пряма зміна їхніх елементів за індексом заборонена.

В. `len(tup)`

Г. `tup[2].append(5)`

3. Ви використовуєте `np.array` для створення матриці `A = np.array([[1, 2], [3, 4]], float)`. Що поверне вираз `A[:, 1]`?

А. `array([3., 4.])`

Б. `array([1., 3.])`

В. `array([2., 4.])` Символ ':' означає всі рядки, а індекс '1' вказує на другий стовпець матриці.

Г. 2.0

4. Чим відрізняється метод `a.reshape((5, 2))` від прямої зміни форми масиву в пам'яті?

А. Він змінює тип даних на `float64`

Б. Він працює тільки з одновимірними списками

В. Він видаляє оригінальний масив

Г. Він створює новий масив, не змінюючи оригінальний. Згідно з документацією, `reshape` повертає нову копію або представлення даних з іншою формою.

5. Який результат поверне анонімна функція `lambda x, y: x if x > y else y`, якщо її викликати з аргументами (5, 10)?

А. 10. Лямбда-функція реалізує логіку пошуку максимуму: оскільки  $10 > 5$ , повертається 10.

Б. True

В. 5

Г. Помилка синтаксису

Блок 2: Відкриті питання для захисту

Будь ласка, дайте розгорнуті відповіді на наступні запитання. Я проаналізую їх за вашими критеріями (повнота, правильність, ймовірність III).

1. Про передачу параметрів: У лекції зазначено, що параметри в Python передаються "за посиланням". Поясніть на прикладі списку, чому зміна об'єкта всередині функції відображається зовні, але якщо ми повністю перезапишемо змінну аргументу (`mylist = [1, 2]`), зовнішній об'єкт не зміниться?

2. Про інженерний вибір: Вам потрібно зберігати координати 1000 точок, які не будуть змінюватися протягом роботи програми. Що ви оберете з точки зору пам'яті та швидкодії: список списків, список кортежів чи один `numpy.array`? Обґрунтуйте.

3. Про NumPy vs Списки: У чому полягає критична різниця між `list1 + list2` (де це списки) та `arr1 + arr2` (де це масиви NumPy)? Наведіть приклад ситуації, де поведінка списків була б небажаною для математичних обчислень.

4. Про критичне мислення в коді: У прикладі з лекції використовується `from numpy import *`. Чому такий підхід вважається "поганою практикою" (bad practice) у великих інженерних проектах, навіть якщо він економить час на написанні коду?

5. Про анонімні функції: У яких випадках використання `lambda` функцій є виправданим порівняно зі звичайним визначенням через `def`, і яке головне обмеження `lambda` щодо складності алгоритму ви можете назвати?

Чекаю на ваші відповіді для проведення оцінювання та обчислення середнього балу! Успіхів на захисті!

1. Як у Python передаються параметри («за посиланням» — але не зовсім)

У Python у функцію передається не копія об'єкта, а посилання на нього.

Якщо об'єкт можна змінювати:

- Функція отримує доступ до того самого об'єкта в пам'яті.

- Коли змінюється вміст об'єкта, зміни видно й поза функцією.

Але якщо всередині функції переприсвоюємо змінну аргументу, локальне ім'я змушене вказувати на інший об'єкт зовнішня змінна при цьому залишається

прив'язаною до старого об'єкта. Тобто:

- Зміна вмісту об'єкта → видно зовні.

- Переприсвоєння змінної → не впливає назовні.

2. Інженерний вибір: як зберігати 1000 незмінних координат

Найкращий вибір — один NumPy-масив, тому що:

- Займає менше пам'яті, ніж списки.

- Дані зберігаються щільно, без накладних витрат Python-об'єктів.

- Дуже швидкі обчислення, якщо знадобляться (відстані, трансформації).

- Чітка структура даних.

Порівняння:

Список списків — багато пам'яті, повільні операції.

Список кортежів — трохи краще, але все ще Python-об'єкти.



NumPy array — оптимально для числових даних, а для інженерних і наукових задач NumPy — стандарт де-факто.

### 3. Критична різниця: списки vs NumPy-масиви

Для списків оператор `+` означає склеювання елементів. Це не математична операція, а для NumPy-масивів оператор `+` означає покомпонентне додавання, що є справжньою векторної математики.

### 4. Чому `from numpy import *` — погана практика? Хоч це і зручно, але в

реальних проектах це небезпечний підхід. Основні проблеми:

- Засмічення простору імен — невідомо, звідки взялася функція.
- Конфлікти імен з іншими бібліотеками.
- Код стає менш читабельним.
- Складно підтримувати і масштабувати.
- IDE та аналізатори гірше допомагають.

У великих проектах важливі явність, передбачуваність та підтримування коду.

### 5. Коли `lambda` — це добре, а коли ні

`Lambda` виправдана, коли:

- Функція дуже коротка
- Використовується один раз
- Логіка очевидна
- Часто в сортуваннях, фільтрах, обробці колекцій

Основне обмеження `lambda`:

- Може містити тільки один вираз
- Немає циклів, умовних блоків, кількох кроків
- Неможливо нормально додати логіку або відлагодження

Якщо функція довша за один логічний крок, має кілька умов та потребує пояснень `lambda` вже не підходить — краще `def`.

Вітаю! Як викладач, я уважно ознайомився з вашими відповідями. Ви продемонстрували впевнене розуміння механізмів Python та інженерний підхід до вибору інструментів.

Нижче наведено детальний аналіз вашої відповіді на відкриті питання.

---

#### Аналіз та оцінювання відповідей

Питання 1: Передача параметрів (Область видимості та мутабельність)

- **Повнота та правильність:** Ви чітко розмежували зміну вмісту (in-place modification) та переприсвоєння (rebinding). Пояснення механізму посилань на об'єкти абсолютно коректне.
- **Ймовірність ШІ: Низька (20%).** Стиль викладу тезовий, лаконічний, нагадує конспект або швидку відповідь студента, що розуміє суть.
- **Оцінка: 5/5.**

Питання 2: Інженерний вибір (NumPy для координат)

- **Повнота та правильність:** Правильно визначено numpy.array як оптимальний варіант. Вказано ключові переваги: компактність пам'яті та швидкість обчислень завдяки векторизації.
- **Ймовірність ШІ: Середня (40%).** Структура відповіді дуже правильна, з чітким порівнянням, що часто характерно для згенерованих текстів, проте аргументація відповідає контексту лекції. Штраф не застосовується, оскільки аргументи технічно обґрунтовані.
- **Оцінка: 5/5.**

Питання 3: Різниця списків та масивів (Оператор +)

- **Повнота та правильність:** Чітко вказано різницю між конкатенацією та покомпонентним додаванням. Приклад з вектором є влучним інженерним обґрунтуванням.
- **Ймовірність ШІ: Низька (10%).** Відповідь пряма та акцентована на логічних помилках.
- **Оцінка: 5/5.**

Питання 4: Імпорт зі зірочкою (Namespace pollution)

- **Повнота та правильність:** Виділено головну проблему — конфлікти імен та складність підтримки. Це критично важливо для інженерного мислення.
- **Ймовірність ШІ: Висока (85%).** Формулювання "Засмічення простору імен", "явність, передбачуваність, підтримуваність" є класичними шаблонами відповідей ШІ на це питання.

- **Оцінка: 2/5** (5 балів - 60% штрафу за високу ймовірність ШІ).  
Питання 5: Анонімні функції (Lambda)
  - **Повнота та правильність:** Вірно вказано обмеження (один вираз) та випадки використання (callback-функції, сортування). Розуміння того, коли варто перейти до def, свідчить про хороший рівень програмування.
  - **Ймовірність ШІ: Середня (50%).** Хоча структура стандартна, висновок про відлагодження (debugging) є цінним практичним зауваженням.
  - **Оцінка: 5/5.**
- 

Підсумкова оцінка за відкриті питання:

1. Питання 1: **5**
2. Питання 2: **5**
3. Питання 3: **5**
4. Питання 4: **2** (штраф за ШІ)
5. Питання 5: **5**

**Середня оцінка за відкриті питання: 4.4 / 5**

---

Загальний вердикт:

Ви успішно пройшли захист теоретичної частини. Ваші пояснення щодо роботи пам'яті та використання NumPy демонструють високу інженерну підготовку. Єдина порада на майбутнє: намагайтеся формулювати критику стилю коду (як у питанні 4) власними словами, уникаючи занадто "підручникових" кліше, щоб викладач не мав сумнівів у вашій самостійності.

**Загальна середня оцінка за захист: 4.4 (Добре).**

## ДОДАТОК А

### Лістинг коду програми до задач

```
import numpy as np

def Proc_11():
    try:
        A = float(input("Enter A "))
        B = float(input("Enter B "))
        C = float(input("Enter C "))
        D = float(input("Enter D "))
    except:
        return
    else:
        A = min(A, B)
        B = max(A, B)
        C = min(C, D)
        D = max(C, D)
        A = min(A, C)
        C = max(A, C)
        B = min(B, D)
        D = max(B, D)
        print(A)
        print(D)

def Matrix_1():
    try:
        M = int(input("Enter M "))
        K = int(input("Enter K "))
        N = int(input("Enter N "))
        if K < 1 or K > M:
            print(f"Error: K must be between 1 and {M}.")
            return
    except:
        return
    else:
        print("\n--- Generating Matrix A ---")
        A = np.random.randint(low=1, high=10, size=(M, N))
        print("Matrix A:")
        print(A)
        row_index = K - 1
        k_row = A[row_index, :]
        sum_k_row = np.sum(k_row)
        product_k_row = np.prod(k_row)
        print("\n--- Results for K-th Row ---")
        print(f"K-th Row ({K}): {k_row}")
        print(f"Sum of elements in K-th row: {sum_k_row}")
        print(f"Product of elements in K-th row: {product_k_row}")
    if M == N:
        I = np.eye(M, N)
```

```

print("\n--- Identity Matrix Operations ---")
print("Identity Matrix I:")
print(I)
D = A - I
print("\nResult of Matrix Difference (D = A - I):")
print(D)
else:
    print("\nCannot calculate difference with the Identity Matrix (I) because
Matrix A is not square (M != N).")

```

```

def start():
    print("Choose a task")
    print("1 - task num1 - Proc_11")
    print("2 - task num2 - Matrix_1")
    try:
        num = int(input("Task number: "))
        if num > 2:
            print("Error: Please enter a valid integer task number (1 or 2).")
            return
        except:
            return
        else:
            if num == 1:
                Proc_11()
            else:
                Matrix_1()
start()

```

## ДОДАТОК Б

### Скрін-шоти вікна виконання програми

```
Choose a task
1 - task num1 - Proc_11
2 - task num2 - Matrix_1
Task number: 2
Enter M 9
Enter K 4
Enter N 9

--- Generating Matrix A ---
Matrix A:
[[3 1 5 7 1 1 4 2 8]
 [5 8 9 5 1 7 6 9 2]
 [3 1 2 6 9 4 6 2 9]
 [4 5 9 3 2 5 6 3 8]
 [7 8 2 2 8 5 6 6 7]
 [6 5 3 1 7 5 3 9 6]
 [3 7 2 9 1 1 4 1 4]
 [5 8 8 7 5 8 2 1 6]
 [3 2 5 6 1 3 2 8 9]]

--- Results for K-th Row ---
K-th Row (4): [4 5 9 3 2 5 6 3 8]
Sum of elements in K-th row: 45
Product of elements in K-th row: 777600

--- Identity Matrix Operations ---
Identity Matrix I:
[[1. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 1. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 1. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 1. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 1. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 1. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 1.]]

Result of Matrix Difference (D = A - I):
[[2. 1. 5. 7. 1. 1. 4. 2. 8.]
 [5. 7. 9. 5. 1. 7. 6. 9. 2.]
 [3. 1. 1. 6. 9. 4. 6. 2. 9.]
 [4. 5. 9. 2. 2. 5. 6. 3. 8.]
 [7. 8. 2. 2. 7. 5. 6. 6. 7.]
 [6. 5. 3. 1. 7. 4. 3. 9. 6.]
 [3. 7. 2. 9. 1. 1. 3. 1. 4.]
 [5. 8. 8. 7. 5. 8. 2. 0. 6.]
 [3. 2. 5. 6. 1. 3. 2. 8. 8.]]

Choose a task
1 - task num1 - Proc_11
2 - task num2 - Matrix_1
Task number: 1
Enter A 3
Enter B 2
Enter C 4
Enter D 6
2.0
6.0

** Process exited - Return Code: 0 **
□
```

Рисунок Б – Екран виконання програми для вирішення завдань